

Chapter 4

Gathering Stories

How do you gather the stories? This chapter offers advice on working with users in order to identify stories in your conversations with them. The advantages of various approaches will be described. This chapter describes effective methods for getting at a user's real needs by asking the right types of questions.

Elicitation and Capture Should Be Illicit

Even some of the best books on requirements use words like *elicitation* (Kovitz 1999; Lauesen 2002; Wiegers 1999) and *capture* (Jacobson, Booch and Rumbaugh 1999) to describe the practice of identifying requirements. Terms like these imply requirements are out there somewhere and all we need to do is have them explained to us and then we can lock them in a cage. Requirements are not out there in the project space waiting to be captured. Similarly, it is not the case that users already know all the requirements and we need only elicit them.

Robertson and Robertson (1999) introduce the term trawling to describe the process of gathering requirements. Trawling for requirements leads to the mental image that requirements are captured in a fishing net being pulled behind a boat. This metaphor works on a variety of levels.

First, it is consistent with the idea that different-sized nets can be used to capture different-sized requirements. A first pass can be made over the requirements pond with a large mesh net to get all the big ones. You can get a feel for the needed software from the big requirements and then make a subsequent pass with a smaller mesh net and get the medium-sized requirements, still leaving the small ones for later. This metaphor works whether we think of size as business value, essentialness to the software, and so on.

Second, trawling for requirements expresses the idea that requirements, like fish, mature and possibly die. My net may miss a requirement today because the requirement is not important to the system. However, as the system grows in

unpredictable directions based on the feedback from each iteration, some requirements will grow in importance. Similarly, other requirements that were once considered important will decrease in importance to the point where we can consider them dead.

Third, just like you won't catch all of the fish in an area by trawling, you won't capture all of the requirements. However, just as with trawling for fish, when you trawl for requirements you are likely to capture the flotsam and jetsam that bloat requirements.

Finally, the metaphor of trawling for requirements captures the important reality that skill plays a factor in finding the requirements. A skilled requirements trawler will know where to look for requirements, while the unskilled trawler will waste time with inefficient techniques or in the wrong locations. This chapter is about learning the techniques that make us efficient in trawling for user stories.

A Little Is Enough, or Is It?

One of the easiest ways to spot a traditional prescriptive process is to look at its approach to requirements. Prescriptive processes are characterized by their heavy emphasis on getting all the requirements right and written early in the project. Agile projects, on the other hand, acknowledge that it is impossible to use a net with such a fine mesh that we can get all of the user stories in one pass. Agile processes also acknowledge that there is a time dimension to stories: the relevance of a story changes based on the passage of time and on what stories were added to the product in prior iterations.

However, even though we acknowledge the impossibility of writing all of the stories for a project, we should still make an initial upfront attempt to write those that we can, even if many are written at a very high level. One of the advantages of working with stories is that it is very easy to write them at different levels of detail. We can write "A user can search for jobs" either as a placeholder or because that's all we know at the time. We can then evolve that story into smaller, more useful stories later. Because of this, it is very easy to write stories for a large portion of an application with less work than with other requirements techniques.

This is not a recommendation to start a new project by spending three months writing user stories. Rather, it means to look into the future for approximately one release (perhaps three to six months) and then write user stories that decrease in detail as the time horizon increases. For example, if the cus-

tomers or users have said they “probably want reports in this release” then write a card that simply says “A user can run reports.” But stop there: don’t determine if they need to configure their own reports, whether reports are formatted in HTML, or whether reports can be saved.

Similarly, it is often important to get an overall feel for the size of an application long in advance of starting it. It is often necessary to have a rough idea of what a project will cost and what benefits it will deliver before getting funding and approval to start the project. To know the answers to these questions requires at least a little forethought into the stories that will comprise the project.

Techniques

Because stories will be evolving, coming and going throughout the project, we need a set of techniques for gathering them that can be used iteratively. The techniques we use must be sufficiently lightweight and non-obtrusive that they can be applied more or less continuously. Some of the most valuable techniques for creating a set of stories are:

- User interviews
- Questionnaires
- Observation
- Story-Writing workshops

Many of these techniques are in the toolkit of the traditional business analyst. Projects with a business analyst available should make use of her to do much of the trawling for stories.

Each of these techniques will be considered in the following sections.

User Interviews

Interviewing users is the default approach many teams take to trawling for stories and is probably one you will want to use. One of the keys to success with interviews is the selection of interviewees. As discussed in Chapter 5, “Working with User Proxies,” there are many user proxies available; but you should obviously interview real users whenever possible. You should also interview users who fill different user roles.

It is not sufficient to ask the user “So, what do you need?” Most users are not very adept at understanding, and especially at expressing, their true needs. I learned this once from a user who walked into my office and acknowledged, “You built exactly what I asked for but it’s not what I want.”

I worked with a team that was developing software for delivering surveys. Each survey would be delivered over the phone, via email, and via interactive voice response. Different types of users would use different survey types. The surveys were very complicated: specific answers to one set of questions would determine which question would be asked next. The users needed a way to enter the surveys and they presented the development team with examples of a complicated mini-language they proposed using to formulate questions. This entirely text-based approach seemed needlessly complicated to one of the developers. The developer showed the users how they could instead create surveys visually by dragging and dropping icons that represented different types of questions in a survey. The users ripped up their mini-language and worked with the developers to create a visual survey design tool. Just because these users had the problem does not mean they were uniquely qualified to propose its solution.

Open-Ended and Context-Free Questions

The best technique for getting to the essence of a user’s needs is through the questions you ask. I worked with a project team that was torn between putting their application in a browser or writing it as a more traditional platform-specific program. They struggled between the ease-of-deployment and lower training costs provided by the browser-based version and the more powerful platform-specific client. The intended users would certainly like the advantages of the browser, but they also valued the richer user experience provided by the platform-specific client.

It was suggested that the target users for the product be asked their preference. Since the product would be a new generation rewrite of a legacy product, the marketing group agreed to contact a representative sample of current users. Each user in the survey was asked “Would you like our new application in a browser?”

This question was like going to your favorite restaurant and having the waiter ask if you’d like to have your meal for free. Of course, you would! And of course the surveyed users responded that they would love to have the new version of the software in a browser.

The mistake the marketing group made was that they asked a closed-ended question and failed to provide sufficient detail for it to be answered. The question assumed that anyone being interviewed would know the tradeoffs between

the browser and the unstated alternatives. A better version of the question would have been:

Would you like our new application in a browser rather than as a native Windows application even if it means reduced performance, a poorer overall user experience, and less interactivity?

This question still has a problem because it is closed-ended. The respondent is given no room for anything other than a simple yes or no. It is far better to ask open-ended questions that let respondents express more in-depth opinions. For example, “What would you be willing to give up in order to have our next generation product run within a browser?” A user answering that question can go in a variety of directions. Where she goes—and does not go—with her answer will provide you with a more meaningful answer to the question.

It is equally important to ask context-free questions, which are ones that do not include an implied answer or preference. For example, you would not ask, “You wouldn’t be willing to trade performance and a rich user experience just for having the software in a browser, would you?” It’s pretty clear how most people are going to answer that question.

Similarly, instead of asking “How fast do searches need to be?” ask “What kind of performance is required?” or “Is performance more important in some parts of the application?” The first question is not context-free because it implies there is a performance requirement to searching. There may not have been but, having been asked, a user is unlikely to say so; she’s more likely to take a guess.

At some point you will need to move from context-free questions to very specific questions. However, by starting with context-free questions you leave open the possibility for a wider range of answers from users. That will lead you to stories that may have remained undiscovered if you jumped right into very specific questions.

Questionnaires

Questionnaires can be an effective technique for gathering information about stories you already have. If you have a large user population, then a questionnaire can be a great way to get information about how to prioritize the stories. Questionnaires are similarly useful when you need answers from a large number of users to specific questions.

However, questionnaires are usually inappropriate as a primary technique of trawling for new stories. Questionnaires do not lend themselves to followup

questions. Also, unlike in a conversation, it is impossible to follow a user down an interesting path that comes up.

As an example use of a questionnaire, you may survey current users about how often they use features in the software today and what their reasons are for not using some feature more. This may lead to prioritizing some usability stories higher than they have been prioritized in the past. As another example, a questionnaire that asked “What new features would you like to see?” will be of limited use. If you give the user a list of choices then you may miss hearing about the five critical features you’ve never thought of. Alternatively, if the user is allowed to respond with free-form text it will be difficult to tabulate answers.

Given their one-way communication nature and inherent time lag, I do not recommend using questionnaires when trawling for stories. If you want to gather information from a broad base of existing users and can wait one or more iterations to incorporate the information, then use them in that manner but not as a primary means of gathering stories.

Observation

Observing users interact with your software is a wonderful way to pick up insights. Every time I have had the chance to observe someone using my software, I have left flush with ideas about how to improve their experience, productivity, or both. Unfortunately, opportunities for user observation are rare unless you are developing for in-house customers. Too many commercial products take the approach that it’s possible to guess what users want. If you have the chance to observe users work with your software, take it. This chance for rapid and direct feedback from users is one of many reasons to release software as early and often as possible.

At one company the users were nurses working in a call center. The nurses answered medical questions from callers. The nurses indicated that they needed a large text field that could be used for documenting the results of the call when the call was finished. An initial version of the software included a large text field on the call wrapup screen. However, after the initial release each member of the development team spent a day observing the users. One of the things discovered was that the large text field was used for entry of things that could have been tracked by the system. By observing the users, the developers discovered that the real need was for the system to track the decisions made by the user as she worked with the software. The large text field was replaced with a feature that logged all searches and recommendations the nurse selected. The real

need—tracking all instructions given to a caller—had been obscured by the nurses’ description of the need and only came out during observation.

Story-Writing Workshops

A story-writing workshop is a meeting that includes developers, users, the product customer and other parties who can contribute by writing stories. During the workshop the participants write as many stories as they can. No priorities are associated with the stories at this point; the customer will have a chance to do that later. In my opinion, a story-writing workshop is the most effective way to quickly trawl for stories. At a minimum, I recommend conducting a story-writing workshop prior to starting each planned release. You can always hold additional workshops while working toward the release but that typically is not necessary.

A properly conducted story-writing workshop can be a very rapid way to write a great number of stories. From my experience, a good story-writing workshop combines the best elements of brainstorming with low-fidelity prototyping. A low-fidelity prototype is done on paper, note cards, or a white board and maps very high level interactions within the planned software. The prototype is built up iteratively during the workshop as the participants brainstorm the things a user may want to do at various points while using the application. The idea is not to identify actual screens and fields, as in traditional prototyping or Joint Application Design (JAD) sessions. Rather, the conceptual workflows are identified. Figure 4.1 shows the start of a low-fidelity prototype for the Big-MoneyJobs website.

Each box represents a new component of the website. The component’s title is underlined in the box. Below the title is a very short list of what the component does or contains. The arrows between the boxes represent links between the components. For a website, a component may be either a new page or space on the current page. So, a link indicates that a new page appears or that the information is displayed on the same web page. For example, Search Jobs may be a page on its own or it may be an area on the home page. The distinction is not important, as the customer and developers will have plenty of time to discuss specifics like that later.

To start a low fidelity prototype, first decide which of the system’s user roles or personas you’d like to start with. You’ll repeat the process using each role or persona so the order does not matter. Next, draw an empty box and tell the participants that it is the main screen of the software, and ask them what the

selected user role or persona can do from there. It does not matter that you haven't figured out what the main screen is yet and what's active on that screen. The meeting participants will start throwing out ideas about what actions the role or persona can take. For each action, draw a line to a new box, label that box, and write a story.

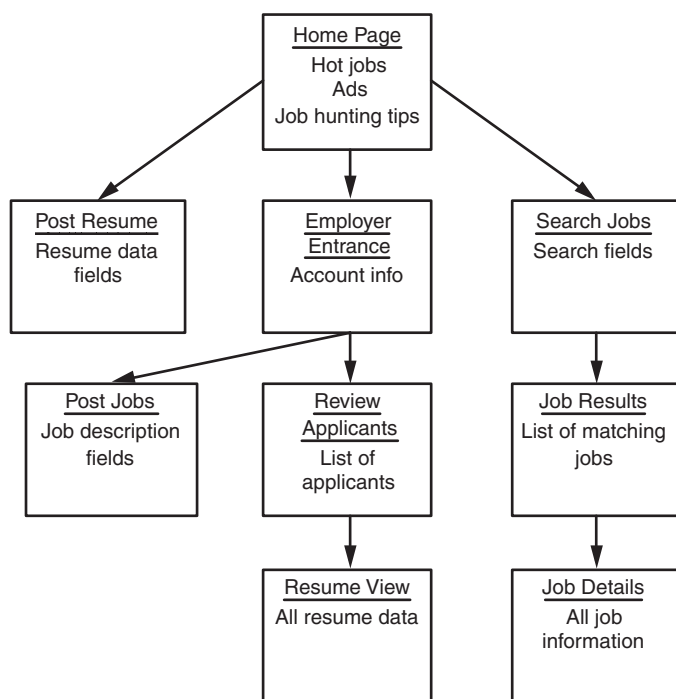
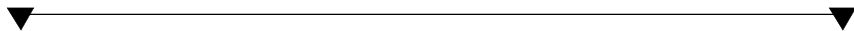


Figure 4.1 A low-fidelity prototype for BigMoneyJobs.

The discussion that led to the creation of Figure 4.1 will generate stories such as:

- A Job Seeker can post her resume.
- An Employer can post jobs.
- An Employer can review submitted resumes.
- A Job Seeker can search for jobs.
- A Job Seeker can view results of jobs that match a search.
- A Job Seeker can view details about specific jobs.

None of these stories requires knowledge about how the screens will be designed. However, walking through the workflows will help everyone involved think of as many stories as possible. I have found it most efficient to take a depth-first approach: For the first component, write down its salient details, then move to a component connected to the first and do the same. Then, move to a component connected to that one, rather than going back to the first component and first describing each component connected to it. Using a breadth-first approach can be somewhat disorienting as it becomes hard to remember where you left off in pursuing each path to its end.



Throw It Away

Be sure to throw away or erase the low-fidelity prototype within a few days of creating it. A prototype is not a long-term artifact of your development process and you don't want to cause any confusion by keeping it around. If you leave a story-writing workshop with the feeling that you didn't finish, then keep the prototype around for a few more days, revisit it, try to write any missing stories, and then get rid of it.

A low-fidelity prototype does not need to go into the trash at the end of the day that you created it. But it needs to end up there soon.



As you walk through the prototype, ask questions that will help you identify missing stories, such as:

- What will the user most likely want to do next?
- What mistakes could the user make here?
- What could confuse the user at this point?
- What additional information could the user need?

Think about the user roles and personas as you ask these questions. Many of the answers can change based on the user role being considered.

Maintain a parking lot of issues to come back to. For example, in discussing BigMoneyJobs someone may ask if the system will support contract workers as well as full-time employees. If no one has thought of that prior to the workshop, write it down some place where you can see it and can come back to it later—either at the end of the workshop or after some followup work after the workshop.

During a story writing workshop the focus should be on quantity rather than quality. Even if you'll eventually keep your stories electronically, during the

story-writing workshop use cards. Just let the ideas come and write them down. A story that seems like a bad idea now might seem brilliant in a few hours or it may be the inspiration for another story. Additionally, you do not want to get bogged down in lengthy debate over each story. If a story is redundant with or becomes replaced by a better story later, then you can just rip up the story. Similarly, when the customer prioritizes stories for a release, she can assign a low priority to low quality stories.

Occasionally some participants in a story-writing workshop have a hard time either getting started or moving past a sticking point. In this case it can be very beneficial to have access to competitive or similar products.

Pay attention to who is contributing during a story writing workshop. Occasionally a participant will remain silent through much or all of the meeting. If this is the case, talk to the participant during one of the breaks and make sure she's comfortable with the process. Some participants are reluctant to speak up in front of their peers or supervisors, which is why it is important that story ideas not be judged during these sessions. Once participants become comfortable that their ideas will simply be noted, not debated, at this point they will contribute more readily.

Finally, let me reiterate that discussion during a user story workshop should remain at a very high level. The goal is to write as many user stories in as short a time as possible. This is not the time to design screens or solve problems.

Summary

- The idea of eliciting and capturing requirements is wrong. It leads to the twin fallacies that users already know all the requirements and that requirements can be captured and locked in a cage where they will remain unchanged.
- The metaphor of trawling for requirements is far more useful: it captures the ideas that there are different sizes of requirements, that requirements may change over time, and that it takes skill to find the requirements.
- While agile processes are supportive of requirements that emerge late in the process, you should still start by looking forward to approximately the end of the intended release and write the user stories you can easily see.
- User stories can be found by interviewing users, observing users, questionnaires, and holding story-writing workshops.

- The best results are achieved by using a combination of methods rather than overreliance on any one method.
- The most useful answers are given in response to open-ended, context-free questions, such as “Tell me about how you’d like to search for a job” rather than “Will you search for a job by title?”

Developer Responsibilities

- You are responsible for understanding and using multiple techniques while trawling for user stories.
- You are responsible for knowing how to best make use of open-ended and context-free questions.

Customer Responsibilities

- You are responsible for understanding and using multiple techniques while trawling for user stories.
- You are responsible for writing as many user stories as early as possible.
- As the main representative of the software’s users, you are responsible for understanding your options in communicating with them.
- You are responsible for knowing how to best make use of open-ended and context-free questions.
- If you need or want help in writing the stories, you are responsible for scheduling and running one or more story-writing workshops.
- You are responsible for making sure all user roles are appropriately represented while trawling for stories.

Questions

- 4.1 What problems would you expect if a team only gathered requirements through the use of questionnaires?

- 4.2 Rephrase the following questions to be context free and open-ended.
Do you think the user should have to enter a password? Should the system automatically save the user's work every 15 minutes? Can one user see database entries saved by another user?
- 4.3 Why is it best to ask open-ended, context-free questions?